refactor_utils

Consolidate a Python package or project into one dependency-ordered file.

Documentation

Version 2.0.0
https://github.com/i4147/refactor_utils

Contents

refactor_utils **5**

What it does 5

Where to go next 5

Requirements 5

License 6

Getting started **7**

Install 7

First run 7

Preview before writing 8

What gets skipped 8

Command line **9**

Source and output 9

Discovery 9

Parsing 9

Entry points 10

Content 10

Output control 10

Logging 11

Exit codes 11

Removed flags 11

Configuration **12**

Keys 12

Precedence 12

Python API **13**

consolidate 13

ConsolidationResult 14

validate_syntax 14

Naming helpers 14

Recipes **15**

Consolidate the current project 15

Stable file name in CI 15

Preview without writing 15

Skip vendored code 15

Include the tests directory 15

Format the result and drop comments 15

Keep a single entry point 15

Code only, no data files 15
See what was renamed 15
Check any Python file with the same validator 16
Pipeline 17
1. Discovery (discovery.py) 17
2. Parsing (parser.py) 17
3. Resolution (resolver.py) 18
4. Ordering (ordering.py) 18
5. Rendering (writer.py) 18
6. Validation and writing (core.py) 18
Name resolution and renaming 19
Identical bodies are merged 19
Different bodies are renamed 19
Generated names are always valid Python 19
Example 20
References are rewritten 20
Scope 20
Imports 21
Internal imports disappear 21
External imports are merged 21
Modes 21
Conflicts 21
Guarded imports 21
TYPE_CHECKING 22
Nested imports 22
Star imports 22
Docstrings, comments and all 23
Module docstrings are pruned 23
Class and function docstrings 23
Comments 23
__all__ 23
Entry points 24
main goes last 24
Guards are merged 24
Choosing entries 24
Validation 25
Guarantees 25
What it does not check 25

Failure 25
Input files 25
Using the validator elsewhere 26
Troubleshooting **27**
N file(s) could not be parsed; refusing to emit a partial consolidation 27
generated X.py is not valid python 27
A class was renamed to m2_Something 27
cannot resolve internal import 27
An import I need was removed 27
The module docstring is gone 27
Only one __main__ block is left 27
The result parses but fails when imported 27
--name did not change the class or function names 28
More detail 28
Known limits **29**
Development **30**
Setup 30
Layout 30
Helper scripts 30
Tests 30
Documentation 31
Adding a rename rule 31
Changelog **32**
Unreleased 32
Fixed 32
Added 32
Changed 32
2.0.0 32

refactor_utils

`refactor_utils` consolidates a small or medium Python module, library or project into **one** dependency-ordered file. It installs the `pyrefactor` command.

```
pip install -e .
pyrefactor path/to/package        # writes ./output/<current-directory-name>.py
```

A run without flags produces exactly one generated Python file:

```
$ cd myproject
$ pyrefactor src/mypackage
output/
  myproject.py        # imports -> constants -> classes -> functions -> entry points
```

What it does

Step	Behaviour
Discover	Walks the source tree, skipping tests, virtualenvs, build output and a few well-known files. Data files (`.json`, `.csv`, ...) are copied next to the result.
Parse	Every file is parsed with tree-sitter in parallel workers and split into top-level blocks.
Resolve	Same-named objects are deduplicated by content hash or renamed to a unique, **valid Python identifier**. Every reference is rewritten.
Order	Blocks are topologically sorted by import-time dependencies: constants, classes, functions, entry points.
Render	Imports are merged, pruned and sorted. Module docstrings and `__all__` are dropped. `__main__` guards are merged into one.
Validate	The result is parsed with `ast`. Code that is not valid Python is never written.

Where to go next

- New here: Getting started
- All flags: Command line
- Using it from Python: Python API
- Why a name was renamed: Name resolution and renaming
- Something went wrong: Troubleshooting

Requirements

- Python 3.10 or newer
- `tree-sitter`, `tree_sitter_python`, `loguru` (installed automatically)
- optional: `black` or `ruff` for `--format`, `tqdm` for `--progress`

License

MIT. Repository: `i4147/refactor_utils`.

Getting started

Install

```
git clone https://github.com/i4147/refactor_utils
cd refactor_utils
python -m venv .venv
.venv/bin/pip install -e .
```

Optional extras:

Extra	Installs	Needed for
`.[format]`	black	`--format`
`.[progress]`	tqdm	`--progress`
`.[dev]`	pytest, ruff	running the test suite
`.[docs]`	mkdocs, mkdocs-material, xhtml2pdf	building this documentation and `docs.pdf`

First run

Given this project:

```
shop/
  cart.py        # class Cart, def total()
  pricing.py     # TAX = 0.2, def total()   <- different body than cart.total
  __init__.py
```

run:

```
cd shop
pyrefactor .
```

`output/shop.py` is created. The file name is the name of the directory `pyrefactor` was started in; override it with `--name`.

The result has a fixed layout:

```
"""Consolidated from ``shop`` by pyrefactor (generated file)."""

import json                     # merged, pruned, sorted imports

# ---------------------------------------------------------------------------
# Constants
# ---------------------------------------------------------------------------

TAX = 0.2

# ---------------------------------------------------------------------------
# Classes
# ---------------------------------------------------------------------------

class Cart: ...

# ---------------------------------------------------------------------------
# Functions
# ---------------------------------------------------------------------------
```

```
def total(): ...                # from cart (first module alphabetically)

def pricing_total(): ...        # renamed: same name, different body
```

A section header is only written when the section has content. A block that depends on a later section (a constant computed by calling a function, for example) is moved down so the file still runs top to bottom.

Preview before writing

```
pyrefactor . --dry-run        # log the plan, write nothing
pyrefactor . --stdout         # print the module instead of writing it
pyrefactor . --diff           # unified diff against the existing output
```

What gets skipped

Skipped by default:

- directories: `tests`, `test`, `testing`, `docs`, `doc`, `.git`, `.venv`, `venv`, `env`, `build`, `dist`, `node_modules`, caches, `site-packages`, IDE folders, and the output directory itself;
- files: `__main__.py`, `conftest.py`, `test_*.py`, `*_test.py`, `setup.py`, `iwgfyrkugf.py`, `_consolidated.py`.

Bring something back with `--include GLOB`, skip more with `-x GLOB`. Precedence is: default excludes < `--include` < `--exclude`.

> **Do not keep an old result inside the source tree**
>
> A previously generated file that lives inside the source directory (and not in the output directory) is treated as source on the next run. Keep results in `output/` or exclude them with `-x`.

Command line

```
usage: pyrefactor [-h] [-o OUTPUT] [--name NAME] [-j WORKERS]
                  [--executor {process,thread}] [--progress] [-x GLOB]
                  [--include GLOB] [--copy-data | --no-copy-data]
                  [--data-glob GLOB] [--entry MODULE] [--entry-keep-all]
                  [--main-name NAME] [--generate-guard]
                  [--imports {prune,keep,strict}]
                  [--keep-docstrings | --no-keep-docstrings]
                  [--keep-comments | --no-keep-comments] [--dry-run] [--check]
                  [--stdout] [--diff] [--force] [--config PATH] [--format]
                  [--formatter {black,ruff}]
                  [--log-level {TRACE,DEBUG,INFO,WARNING,ERROR}]
                  [--log-file LOG_FILE] [--version]
                  source
```

Source and output

Flag	Default	Effect
`source`	required	package directory, project root or a single `.py` file
`-o, --output DIR`	`output`	output directory
`--name NAME`	current directory name	name of the generated file (`.py` is appended when missing)

Discovery

Flag	Default	Effect
`-x, --exclude GLOB`	none	extra glob to skip, relative to the source; repeatable
`--include GLOB`	none	re-include something excluded by default (`tests/*`, `conftest.py`, ...); repeatable
`--copy-data / --no-copy-data`	on	copy data files to the output, keeping relative paths
`--data-glob GLOB`	none	only copy data files that match; repeatable

Parsing

Flag	Default	Effect
`-j, --workers N`	CPU count	parallel parsing workers
`--executor`	`process`	`process` or `thread`
`--progress`	off	tqdm progress bar; falls back to log lines when tqdm is missing

Entry points

Flag	Default	Effect
`--entry MODULE`	none	keep only the `__main__` guard of this module; repeatable
`--entry-keep-all`	off	keep every guard even when `--entry` is given
`--main-name NAME`	none	rename `main` to `NAME`; must be a valid Python identifier
`--generate-guard`	off	add `if __name__ == "__main__": main()` when no guard exists

Content

Flag	Default	Effect
`--imports`	`prune`	`prune` drops unused imports, `keep` keeps all, `strict` prunes and warns
`--keep-docstrings / --no-keep-docstrings`	on	keep docstrings of classes and functions. Module docstrings are always pruned
`--keep-comments / --no-keep-comments`	on	keep comments
`--format`	off	run a formatter on the result
`--formatter`	`black`	`black` or `ruff`

Output control

Flag	Default	Effect
`--dry-run`	off	log the plan, write nothing
`--check`	off	exit 2 when the existing output would change
`--stdout`	off	print the generated module instead of writing it
`--diff`	off	print a unified diff against the existing output
`--force`	off	silence the overwrite warning
`--config PATH`	auto	`pyproject.toml` with a `[tool.pyrefactor]` table

Logging

Flag	Default	Effect
`--log-level`	`INFO`	`TRACE`, `DEBUG`, `INFO`, `WARNING` or `ERROR`
`--log-file PATH`	none	also log to a file
`--version`		print the version

Exit codes

Code	Meaning
0	success (including a successful `--dry-run` or `--check`)
1	unexpected internal failure (traceback is logged)
2	consolidation error: missing source, unparsable input file, `--check` mismatch, invalid `--main-name`, or a generated module that is not valid Python

Removed flags

Removed in 2.0: `--mode`, `--validate`, `--validate-import`, `--allow-syntax-errors`, `--max-skipped`, `--source-map`, `--preserve-modules`, `--namespace`, `--all`, `--respect-all`, `--rename-strategy`, `--dedupe`, `--no-parallel`, `--jobs`, `--progress-bar`, `--overwrite`, `--keep-all-mains`, `--keep-unused-imports`, `--post-format`, `--pyproject`, `--setup-cfg`, `--strict`, `--fail-on-warning`.

There is one layout, one dedupe strategy (content hash), one rename strategy (`<module>_<name>`) and one validation step (syntax check with `ast`), which always runs and needs no flag. See Validation.

Configuration

`pyrefactor` reads a `[tool.pyrefactor]` table from `pyproject.toml`. The file is looked up next to the source, or given explicitly with `--config PATH`.

```
[tool.pyrefactor]
exclude   = ["vendor/*", "scripts/*"]
include   = []
entry     = ["cli"]
imports   = "prune"        # prune | keep | strict
format    = false
formatter = "black"        # black | ruff
copy_data = true
data_glob = ["*.json"]
```

Keys

Key	Type	CLI equivalent
`exclude`	list of globs	`-x / --exclude`
`include`	list of globs	`--include`
`entry`	list of module names	`--entry`
`imports`	string	`--imports`
`format`	bool	`--format`
`formatter`	string	`--formatter`
`copy_data`	bool	`--copy-data / --no-copy-data`
`data_glob`	list of globs	`--data-glob`

A single string is accepted wherever a list is expected.

Precedence

- `exclude`, `include`, `entry`: the command line wins when it passes at least one value; otherwise the config value is used.
- `imports`, `formatter`, `copy_data`, `data_glob`: when the key is present in the config file, **the config value is used**, also if a flag was passed.
- `format = true` in the config enables formatting; it cannot be switched off from the command line.

> **Note**
>
> To override a config value for a single run, point `--config` at a different `pyproject.toml`.

Python API

```
from refactor_utils import (
    ConsolidationError,
    ConsolidationResult,
    consolidate,
    validate_syntax,
)
```

consolidate

```
from pathlib import Path
from refactor_utils import ConsolidationError, consolidate

try:
    result = consolidate(
        Path("src/mypackage"),
        Path("output"),
        filename="mypackage",
        imports_mode="prune",
        copy_data=False,
    )
except ConsolidationError as exc:
    raise SystemExit(f"failed: {exc}")

print(result.files)       # [PosixPath('output/mypackage.py')]
print(result.exports)     # number of public names
for module, old, new in result.plan.renamed:
    print(f"{module}: {old} -> {new}")
```

Keyword arguments mirror the command line:

Argument	Default	CLI flag
`filename`	current directory name	`--name`
`workers`	CPU count	`-j`
`executor`	`"process"`	`--executor`
`exclude`, `include`	`()`	`-x`, `--include`
`imports_mode`	`"prune"`	`--imports`
`entry`	`()`	`--entry`
`entry_keep_all`	`False`	`--entry-keep-all`
`main_name`	`None`	`--main-name`
`generate_guard`	`False`	`--generate-guard`
`keep_docstrings`	`True`	`--keep-docstrings`
`keep_comments`	`True`	`--keep-comments`
`do_format`, `formatter`	`False`, `"black"`	`--format`, `--formatter`
`copy_data`, `data_glob`	`True`, `()`	`--copy-data`, `--data-glob`
`dry_run`, `check`, `stdout`, `diff`, `force`	`False`	same names
`config`	`None`	`--config`
`progress`	`False`	`--progress`

ConsolidationResult

Field	Meaning
`output_dir`	output directory
`files`	generated files (empty for `dry_run`, `check`, `stdout`)
`modules`	number of consolidated modules
`exports`	number of public names in the result
`warnings`	parser warnings
`diff`	unified diff text when `diff=True`
`stdout_text`	the generated module when `stdout=True`
`copied_data_files`	data files copied to the output
`validated`	`True` once the syntax check passed
`dry_run`	`True` when nothing was written
`plan`	the `Plan`: rendered blocks, `renamed`, `deduped`, `exports`

validate_syntax

```
validate_syntax(text: str, filename: str = "<consolidated>") -> ast.Module
```

Parses `text` with `ast.parse`. Returns the tree, or raises `ConsolidationError` naming the file, the line and the offending source line. The code is never executed or imported.

```
from refactor_utils import ConsolidationError, validate_syntax

validate_syntax("x = 1\n")                      # ok

try:
    validate_syntax("class 2_Tokenizer:\n    pass\n", "result.py")
except ConsolidationError as exc:
    print(exc)
# generated result.py is not valid python: invalid decimal literal (line 1): 'class 2_Tokenizer:'
```

`consolidate` calls it on every generated module before writing. See Validation.

Naming helpers

`refactor_utils.resolver` exposes the two functions used for every generated name:

```
from refactor_utils.resolver import is_valid_identifier, make_identifier

make_identifier("2")         # 'm2'
make_identifier("my-mod")    # 'my_mod'
make_identifier("class")     # 'class_'
is_valid_identifier("2_x")   # False
```

Recipes

Consolidate the current project

```
pyrefactor .
```

Writes `./output/<cwd>.py`.

Stable file name in CI

```
pyrefactor src/mypackage --name mypackage --check
```

`--check` exits with code 2 when the committed output is stale.

Preview without writing

```
pyrefactor src/mypackage --dry-run
pyrefactor src/mypackage --stdout | less
```

Skip vendored code

```
pyrefactor . -x "vendor/*"
```

Include the tests directory

```
pyrefactor . --include "tests/*"
```

Format the result and drop comments

```
pyrefactor . --format --formatter ruff --no-keep-comments
```

Keep a single entry point

```
pyrefactor . --entry cli
```

Only the `if __name__ == "__main__":` guard of the module `cli` survives.

Code only, no data files

```
pyrefactor . --no-copy-data
```

See what was renamed

```
pyrefactor . --dry-run 2>&1 | grep Renamed
```

```
INFO | Renamed 2:Tokenizer -> m2_Tokenizer
```

Check any Python file with the same validator

```
python tools/check_syntax.py output/
```

Pipeline

```
cli.py -> core.py orchestrates
        -> discovery.py   find sources and data files, read pyproject config
        -> parser.py      tree-sitter, parallel; blocks, imports, guards
        -> resolver.py    dedupe, renames, import resolution, ordering input
        -> ordering.py    topological sort with deterministic tie-breaks
        -> imports.py     merge, sort and render the import section
        -> writer.py      render the single consolidated file
        -> core.validate_syntax   ast check, then write
models.py          picklable dataclasses shared by parser and resolver
logging_setup.py   loguru configuration
```

1. Discovery (`discovery.py`)

`discover()` walks the source and returns one `SourceFile` per Python file with its dotted module name. The base of the dotted name is detected automatically: a directory with `__init__.py` is a package (its parent is the base), a `src/` layout is recognised, otherwise the directory itself is the base.

`discover_data_files()` returns the non-Python files worth keeping (`.json`, `.yaml`, `.toml`, `.ini`, `.csv`, `.txt`, `.md`, `.sql`, ... and well-known names such as `LICENSE` or `requirements.txt`).

2. Parsing (`parser.py`)

Each file is parsed by tree-sitter in a worker process (or thread). A file with a syntax error aborts the whole run: a partial consolidation is never emitted.

Top-level statements become `Block` objects:

Kind	Source construct
`const`	assignments, type aliases
`class`	class definitions (with decorators)
`func`	function definitions (with decorators)
`misc`	other statements: calls, loops, `with`, mixed compound blocks
`main`	`if __name__ == "__main__":`

Not turned into blocks:

- imports: collected as `ImportSpec`;
- `try/except ImportError` and `if sys.platform ...` import guards: kept verbatim as `GuardBlock`;
- `if TYPE_CHECKING:`: flattened, its imports join the normal imports and its definitions become ordinary blocks;
- `__all__` statements: read as hints, never emitted;
- module docstrings and other bare string statements: pruned.

Every identifier that might refer to a module-level name is recorded as an `Occ` (byte range, name, eager or lazy). The resolver uses these to rewrite references without touching tree-sitter nodes, which keeps worker payloads picklable.

3. Resolution (`resolver.py`)

1. Build one environment per module: local definitions, imports, star imports.
2. Analyse each block: which project blocks and which external imports it uses, and whether the use is eager (import time) or lazy (inside a function body).
3. Deduplicate byte-identical definitions.
4. Assign final names. See Name resolution and renaming.
5. Resolve conflicts between external imports that bind the same local name.
6. Rewrite every occurrence and render the blocks.

4. Ordering (`ordering.py`)

Blocks are sorted topologically on their eager dependencies, preferring constants, then classes, then functions, then module order and source order. A `main` function nobody depends on and the merged `__main__` guard go last. Cycles are reported with a warning.

5. Rendering (`writer.py`)

The file is assembled as: generated docstring, `__future__` imports, imports, verbatim import guards, then the sections Constants, Classes, Functions and Entry points. Optionally a formatter is applied.

6. Validation and writing (`core.py`)

`validate_syntax()` parses the rendered text with `ast`. Only then is the file written, through a temporary file that is renamed over the target, and the data files are copied.

Name resolution and renaming

All modules end up in one namespace, so two modules that define the same name collide. The resolver handles every collision in one of two ways.

Identical bodies are merged

Definitions with the same kind, the same name and the same content hash are emitted once. The hash ignores blank lines and trailing whitespace. The copy from the shallowest module (then alphabetical order) is kept.

```
INFO | Dropping duplicate func 'slugify' from pkg.utils.text (identical to pkg.text)
```

Different bodies are renamed

When the bodies differ, both definitions are kept:

- the first module keeps the plain name. Modules are ordered by depth (fewest dots first), then alphabetically;
- every other definition becomes `<module_prefix>_<name>`.

The module prefix is the dotted module path with dots replaced by _. When the project has a single top-level package, that package name is left out:

Module	Name	Result
`pkg.cart`	`total`	`total`
`pkg.pricing`	`total`	`pricing_total`
`pkg.io.json`	`load`	`io_json_load`
`pkg.pricing`	`_cache`	`_pricing_cache` (leading underscores stay in front)
`pkg.pricing`	`__version__`	`__pricing_version__`

If the candidate is already taken, `_2`, `_3`, ... is appended.

A name is also renamed when it would shadow an external import that another module uses under the same local name, or a name bound by an import guard.

Generated names are always valid Python

File names are not required to be identifiers. `2.py`, `my-mod.py` or `class.py` are perfectly good file names, but `2_Tokenizer`, `my-mod_run` and a bare `class` are not names Python accepts.

Every generated name therefore goes through `make_identifier()`:

Rule	Example
characters that cannot be part of an identifier become _	`my-mod` -> `my_mod`
a name that would start with a digit gets an `m` prefix	`2` -> `m2`
a Python keyword gets a trailing _	`class` -> `class_`

Rule	Example
an empty result falls back to `mod`	`---` -> `mod`

The `m` prefix (rather than _) keeps the renamed object public, so it still counts as an exported name.

Example

`1.py` and `2.py` both define `Tokenizer`, `Encoding` and `__version__` with different bodies:

```
INFO | Renamed 2:Encoding -> m2_Encoding
INFO | Renamed 2:Tokenizer -> m2_Tokenizer
INFO | Renamed 2:__version__ -> __m2_version__
```

```
class Tokenizer: ...           # from 1.py

class m2_Encoding: ...         # from 2.py

class m2_Tokenizer:            # from 2.py
    def encode(self, text):
        return m2_Encoding(...)    # reference rewritten
```

Before this rule existed the second class was written as `class 2_Tokenizer:`, which is a syntax error.

The same guarantee applies to:

- aliases created for conflicting external imports (`from a import load` + `from b import load` -> `from b import load as b_load`);
- names derived from `--main-name`. The value itself is checked up front and rejected with exit code 2 when it is not an identifier.

As a last line of defence, the validation step refuses to write any file that does not parse.

References are rewritten

After names are assigned, every recorded occurrence is rewritten:

- plain names: `Tokenizer(...)`;
- imports with aliases: `from .two import Tokenizer as T` then `T(...)`;
- attribute access through a module: `two.Tokenizer(...)`;
- annotations, decorators, base classes and default values.

Strings are not rewritten: `"Tokenizer(...)"` inside a `__repr__` keeps the original text.

Scope

Renaming is name-based. Function parameters and class bodies are treated as scopes; other local variables are not. See Known limits.

Imports

Internal imports disappear

An import of a module that is part of the consolidation (relative, or absolute below a known top-level package) is resolved to the definition it points to and removed. References are rewritten to the final name.

An internal import that cannot be resolved (the module was excluded or does not exist) is reported:

```
WARNING | 1: cannot resolve internal import 'tokenizers.Tokenizer' (module skipped, missing or excluded?)
```

External imports are merged

External imports from all modules are collected into one section:

- `from __future__` imports first, merged into a single line;
- standard library, then everything else, separated by a blank line;
- inside a group: `import x` lines, then `from x import ...` lines, sorted case-insensitively by module;
- imported names starting with _ sort before the others;
- lines longer than 88 columns are wrapped in parentheses, one name per line.

Modes

`--imports`	**Behaviour**
`prune` (default)	only imports used by a surviving block are emitted
`keep`	every external import is emitted
`strict`	like `prune`, plus a warning for each dropped import

Conflicts

When two different imports bind the same local name, one keeps the name and the others get an alias built from the module path; the references in the affected modules are rewritten. Aliases are valid identifiers, see Name resolution and renaming.

Guarded imports

```
try:
    import ujson as json
except ImportError:
    import json
```

`try/except ImportError` and `if sys.platform ...` blocks that only contain imports are kept verbatim after the import section. Imports they already perform are not hoisted a second time, and names they bind are never reused for a renamed definition.

TYPE_CHECKING

The body of `if TYPE_CHECKING:` is flattened: its imports join the normal import section, and constants, classes and functions defined there become ordinary blocks.

Nested imports

An import inside a function or class stays where it is. If it refers to an internal module it is removed and the references are rewritten.

Star imports

- internal: expanded to the names the module exports (its `__all__` when declared, otherwise the public names);
- external: kept as `from x import *`, with a warning.

Docstrings, comments and all

Module docstrings are pruned

The docstring of a source module describes a file that no longer exists in the result. It is never written to the consolidated file. The result has exactly one module docstring, the generated one:

```
"""Consolidated from ``mypackage`` by pyrefactor (generated file)."""
```

Pruned:

Source	Why
a string as first statement, also after a shebang, an encoding line or comments	module docstring
implicitly concatenated strings, raw strings, single or triple quotes	same thing, different spelling
a bare string after `from __future__ import ...` or anywhere else at module level	no runtime effect; it would otherwise be emitted as a constant
a bare string directly inside `if TYPE_CHECKING:`	same

Kept:

- docstrings of classes, functions and methods (unless `--no-keep-docstrings`);
- f-string statements such as `f"{setup()}"`: their expressions are evaluated, so they are code, not documentation;
- strings assigned to a name: `__doc__ = "..."`, `USAGE = """..."""`.

> **Changed behaviour**
>
> Earlier versions only skipped a docstring when it was a single plain string in first position. Concatenated docstrings and later bare strings were emitted in the Constants section.

Class and function docstrings

`--no-keep-docstrings` removes the docstrings of the emitted blocks. `--keep-docstrings` (the default) keeps them.

Comments

Comment lines directly above a block travel with that block, as does a trailing comment on its last line. `--no-keep-comments` strips comments. Comments attached to a pruned docstring are dropped with it.

`__all__`

`__all__` is never emitted as code. `__all__ = [...]`, `__all__ += [...]`, `__all__.append(...)` and annotated forms are read as hints (they define what an internal star import expands to) and then dropped. The exported surface reported in the log and in `ConsolidationResult.exports` is recomputed from the public names that survive.

Entry points

`main` goes last

A function named `main` that nothing else depends on, and every `if __name__ == "__main__":` guard, are placed in the final Entry points section.

Guards are merged

Several modules often have their own guard. Written one after the other, they would all run, in an order nobody chose. They are merged into a single guard whose body runs each original entry in module order, annotated with its origin:

```
if __name__ == "__main__":
    # from pkg.cli
    main()

    # from pkg.tools.export
    export_all()
```

Guards with an identical body are emitted once. A warning reminds you that several entry points were merged.

Choosing entries

Flag	Effect
`--entry MODULE`	keep only the guard of this module (repeatable; matches the full dotted name or its last part)
`--entry-keep-all`	keep every guard even with `--entry`
`--main-name NAME`	rename `main` to `NAME`
`--generate-guard`	when no guard exists but a `main` function does, add one

`--main-name` must be a valid Python identifier and not a keyword. When several modules define `main`, the first one gets `NAME` and the others `NAME_<module_prefix>`.

Validation

Before anything is written, the generated module is checked by one small function:

```
def validate_syntax(text: str, filename: str = "<consolidated>") -> ast.Module:
    ...
```

It calls `ast.parse(text)`. That is all.

Guarantees

- **Always on.** There is no flag. It runs for normal runs, `--stdout`, `--dry-run`, `--check` and `--diff`.
- **Before writing.** On failure nothing is created or overwritten: not the output directory, not the Python file, not the data files.
- **After formatting.** With `--format` the formatted text is what gets checked.
- **Syntax only.** The code is parsed, never compiled into a module, executed or imported.

What it does not check

Because nothing is executed, these do **not** fail validation:

- a third-party dependency that is not installed;
- module-level code that raises (for example a `@dataclass` with fields that lack annotations);
- undefined names, wrong arguments, type errors.

This is intentional. The former validator also smoke-imported the result, which made runs fail for reasons unrelated to the consolidation itself. It has been removed together with the `--validate`, `--validate-import` and related flags.

If you want the stricter check, run it yourself:

```
python -c "import runpy; runpy.run_path('output/myproject.py')"
```

Failure

```
ERROR | generated result.py is not valid python: invalid decimal literal (line 590): 'class 2_Tokenizer:'
```

The process exits with code 2. From Python, `consolidate()` raises `ConsolidationError` with the same message.

A failure here means `pyrefactor` produced broken code from valid input. That is a bug in the tool: please report it together with the message.

Input files

Source files are checked too, earlier and by tree-sitter: a source file with a syntax error aborts the run and is listed in the error message.

Using the validator elsewhere

```
from refactor_utils import validate_syntax
validate_syntax(open("some_file.py").read(), "some_file.py")
```

or from the shell:

```
python tools/check_syntax.py some_file.py some_directory/
```

Troubleshooting

`N file(s) could not be parsed; refusing to emit a partial consolidation`

A source file has a syntax error. The log names the file and the line:

```
ERROR | /path/mytest/result.py: syntax error near line 590
```

Fix the file, or exclude it with `-x`. A common cause is an old, broken generated file that still lives inside the source directory.

`generated X.py is not valid python`

The syntax check rejected the result, and nothing was written. This is a bug in `pyrefactor`. See Validation.

A class was renamed to `m2_Something`

Two modules define `Something` with different bodies, and the second module's file name starts with a digit (`2.py`). See Name resolution and renaming.

`cannot resolve internal import`

A module imports from a sibling that was excluded, skipped or does not exist. The import is dropped; references to the name are left as they are and may be undefined at run time. Include the missing module (`--include`) or fix the import.

An import I need was removed

`--imports prune` drops imports that no surviving block references by name. Imports needed only for their side effects are affected. Use `--imports keep`.

The module docstring is gone

That is by design: see Docstrings. To keep the text, assign it to a name, for example `USAGE = """..."""`.

Only one `__main__` block is left

Guards are merged into one. Use `--entry MODULE` to select which entry point survives. See Entry points.

The result parses but fails when imported

Validation is syntax-only. Typical causes:

- the original code relied on `__file__`, `__name__` or `__package__` (a warning is logged);
- dynamic imports through `importlib` or `__import__` (a warning is logged);
- the original module itself fails at import time.

`--name` did not change the class or function names

`--name` is only the output file name. Use `--main-name` to rename `main`.

More detail

```
pyrefactor . --log-level DEBUG --log-file pyrefactor.log
```

Known limits

- **Renaming is name-based.** Parameters and class bodies are scoped, other local variables are not. A local variable that has the same name as a renamed module-level object may be rewritten too. A full scope analysis is future work.
- **Strings are not rewritten.** Names inside string literals, including string annotations and `getattr(module, "name")`, keep their original spelling.
- **Single-file semantics.** `__file__`, `__name__` and `__package__` mean something different in one file. A warning is logged; users of `importlib.resources` may need `files()`.
- **Guarded imports** (`try/except ImportError`, `if sys.platform ...`) are kept verbatim and may have side effects.
- **Dynamic imports** through `importlib` or `__import__` are warned about, not resolved.
- **Compound module-level blocks** that mix imports and definitions are emitted verbatim; names bound inside are not rewritten.
- **Validation is syntax-only.** A result that parses can still fail at import time. See Validation.
- **Unresolvable internal imports** are dropped with a warning; the names they provided stay unresolved.

Development

Setup

```
python -m venv .venv
.venv/bin/pip install -e ".[dev,docs]"
```

Layout

```
src/refactor_utils/   the package
tests/                pytest suite
mytest/               sample input (1.py, 2.py) and its consolidated result.py
docs/                 this documentation (MkDocs)
mkdocs.yml            MkDocs configuration and navigation
docs.pdf              the whole documentation as a single PDF
tools/                helper scripts
```

Helper scripts

All scripts can be run from any directory. They use `.venv/bin/python` when it exists and fall back to `python3`.

Script	Purpose
`tools/run_checks.sh`	run the test suite, check the sample and build the docs in strict mode. `--lint` adds ruff, `--no-docs` skips the docs
`tools/check_syntax.py PATH...`	validate Python files or directories with the same `validate_syntax` used by `pyrefactor`; exit code 1 when any file is invalid
`tools/regen_sample.py`	re-create `mytest/result.py` from `mytest/1.py` and `mytest/2.py`. `--check` only verifies it is up to date
`tools/build_docs.sh`	build the HTML site into `site/` and regenerate `docs.pdf`
`tools/build_pdf.py`	build `docs.pdf` from the `nav` in `mkdocs.yml`. `-o PATH` changes the target
`tools/serve_docs.sh`	live preview with `mkdocs serve` (default `0.0.0.0:8000`)

Tests

```
tools/run_checks.sh
# or just
.venv/bin/python -m pytest tests -q
```

`tests/test_consolidate.py` holds the end-to-end suite. `tests/test_naming_and_validation.py` covers identifier validity of generated names, the syntax validator and docstring pruning.

Documentation

```
tools/serve_docs.sh       # preview
tools/build_docs.sh       # site/ and docs.pdf
```

Add a page by creating the Markdown file under `docs/` and listing it in the `nav` of `mkdocs.yml`. The PDF follows the same `nav`, so it needs no extra configuration. Stick to plain Markdown, tables, fenced code and admonitions; those render in both outputs.

`site/` is build output and is ignored by git. `docs.pdf` is committed.

Adding a rename rule

Every name the tool invents must pass through `refactor_utils.resolver.make_identifier`. Add a case to `test_make_identifier` and an end-to-end case next to `test_numeric_module_names_produce_valid_renames`.

Changelog

Unreleased

Fixed

- Collision renames could produce names that are not valid Python. With modules `1.py` and `2.py` both defining `Tokenizer`, the second class was written as `class 2_Tokenizer:`. Generated names now always pass through `make_identifier()`: `m2_Tokenizer`. The same applies to module names with dashes or other punctuation, to keywords, to aliases generated for conflicting imports and to names derived from `--main-name`.
- `--main-name` is rejected with exit code 2 when it is not an identifier.
- The package could not be imported on Python 3.10 to 3.13 because `models.py` used a forward reference in an annotation without `from __future__ import annotations`.

Added

- `validate_syntax(text, filename)`: a small `ast`-based validator. It runs on every generated module before anything is written. It replaces the old compile-and-import validator, which was too strict and had been disabled.
- Documentation under `docs/` (MkDocs), `docs.pdf`, and helper scripts under `tools/`.

Changed

- Module docstrings of source files are pruned in every form (concatenated strings, strings after `from __future__`, stray bare strings). Previously some of them were emitted in the Constants section.
- `mytest/result.py` was regenerated with the fixed tool.

2.0.0

- One layout, one dedupe strategy (content hash), one rename strategy (`<module>_<name>`).
- Removed the `--mode`, `--validate*`, `--namespace`, `--rename-strategy` and related flags. See Command line.